bcc09 for BASIC Programmers

C on the Tandy Color Computer and the Dragon, for those who know Color BASIC

bcc09 project

November 2026

Contents

1 Why C? **2**

2 Your first program **2**

3 PRINT **4**

4 Variables and numbers **5**

5 INPUT and INKEY$ **6**

6 Deciding and repeating **7**

7 GOSUB: functions **9**

8 Arrays **10**

9 Strings **10**

10 PEEK, POKE and the machine **11**

11 A complete program **12**

12 A game **14**

13 Cartridges and disks **15**

14 Where next **15**

Appendix: BASIC to C **15**

1 Why C?

If you have written programs in Color BASIC, you already know how to tell a computer what to do, step by step. C does the same job in a different way:

- **It is compiled.** BASIC reads each line of your program as it runs it, every time. A C compiler translates the whole program into the 6809's own machine language once, on your PC, and the CoCo or Dragon runs that. The result is many times faster than BASIC, often fast enough for games and graphics that BASIC could never manage.
- **It has no line numbers.** A C program is a text file you edit with any editor. Instead of `GOTO 100`, you say what to repeat and when to stop; instead of `GOSUB 1000`, you call a *function* by its name.
- **You say what kind of value each variable holds.** BASIC has numbers and strings; C has small and large whole numbers, numbers with fractions, single characters, and more, and you choose. It is more to think about, and it is also why C programs are small and fast.

This guide takes you from BASIC to C with *bcc09*, a C compiler for the 6809. Each chapter starts from something you know in BASIC and shows how C does it. It doesn't try to teach all of C: when you want more, any book on C will do (the C here is the C of the 1980s and 1990s, "ANSI C"), and the bcc09 user's manual has the details of this compiler and its library.

1.1 What you need

- **bcc09**, installed on a PC running Linux. On Fedora, install the RPM from bcc09's releases on GitHub; otherwise build it from its source, as its README says.
- **A CoCo or a Dragon,** or an emulator of one. *XRoar* emulates the Color Computer 1, 2 and 3 and the Dragon 32 and 64, and is the easiest way to start: you can load a program into it without any cables.
- **A way to get programs to the machine:** a cassette image, which XRoar loads directly and which a real machine can load through its cassette port from a PC's sound card, or a disk image for Disk BASIC.

The examples are for Color BASIC's machines: the CoCo 1 and 2, and the Dragon 32 and 64.

2 Your first program

In BASIC:

```
10 PRINT "HELLO, WORLD"
```

In C, in a file called `hello.c`:

```
#include <stdio.h>
```

```c
int main()
{
    printf("HELLO, WORLD\r");
    return 0;
}
```

Line by line:

- `#include <stdio.h>` brings in the declarations of the standard input and output functions, such as `printf`. Nearly every program starts with a few `#include` lines.
- `int main()` begins the *function* called `main`. Every C program has one: it is where the program starts. The `{` and `}` enclose what the function does.
- `printf("HELLO, WORLD\r");` prints. Every statement in C ends with a semicolon. The `\r` at the end is a *carriage return*: it ends the line, as BASIC's `PRINT` does by itself. (Most C books end lines with `\n`; Color BASIC's screen ends them with `\r`, and bcc09 passes what you print to the screen as it is.)
- `return 0;` ends the program, and goes back to BASIC.

2.1 Compiling and running

On the PC:

```
bcc09 -Me -O -o hello -X-T0x3000 hello.c
objcopy09 -O cas -n HELLO hello hello.cas
```

The first command compiles `hello.c` into a program called `hello`:

- `-Me` says it is for Color BASIC (run with `EXEC`);
- `-O` makes the code smaller and faster;
- `-X-T0x3000` puts the program at address `$3000` in the machine's memory, out of the way of BASIC.

The second command turns `hello` into a cassette image, `hello.cas`, with the name `HELLO`. Now, in XRoar:

```
xroar -load-tape hello.cas
```

and type, at BASIC's `OK`:

```
CLOADM:EXEC
```

`HELLO, WORLD` appears, and you are back at `OK`. For a disk, use `objcopy09 -O decb hello HELLO.BIN` instead, copy `HELLO.BIN` to the disk, and `LOADM"HELLO":EXEC`.

Every program in this guide is compiled and loaded the same way: only the file names change.

2.2 When something is wrong

If the compiler finds a mistake, it says where, and makes no program:

```
hello.c:6.7: error: need ';'
```

means line 6, column 7, of `hello.c`: here, the semicolon is missing from the end of line 5, and the compiler noticed when it came to the next. Mistakes in C are often a missing semicolon or brace, or a misspelled name; look at the line it names, and the one before.

3 PRINT

`printf` prints its text, but it can also print numbers and other values. Where the text has a `%d`, it prints the next value after the text, as a whole number:

```
#include <stdio.h>

int main()
{
    int score;

    score = 1200;
    printf("SCORE: %d\r", score);
    printf("%d + %d = %d\r", 2, 3, 2 + 3);
    return 0;
}
```

prints

```
SCORE: 1200
2 + 3 = 5
```

Some of `printf`'s *conversions*:

In the text	Prints
`%d`	a whole number (an `int`)
`%ld`	a large whole number (a `long`)
`%f`, `%g`	a number with a fraction (a `float` or `double`)
`%c`	a character
`%s`	a string
`%x`, `%X`	a whole number, in hexadecimal
`%%`	a `%` sign

A number between the `%` and the letter gives a width, as `PRINT USING` does: `%5d` prints a number in 5 places, right-aligned, and `%-5d` left- aligned. `%.2f` prints two decimal places.

BASIC's `PRINT A;B` runs values together; in C you write exactly what you want, spaces included, in the text. A `PRINT` ending in `;`, which doesn't end the line, is a `printf` without the `\r`.

3.1 CLS and PRINT@

`coco.h` has BASIC's screen commands:

```
#include <stdio.h>
#include <coco.h>

int main()
{
    cls(-1);
    locate(7, 10);
    printf("IN THE MIDDLE");
    locate(15, 0);
    return 0;
}
```

- `cls(c)` is `CLS c`: `cls(0)` clears the screen to black, `cls(1)` to `cls(8)` to a colour, and `cls(-1)` is a plain `CLS`.
- `locate(row, col)` puts the cursor at a row (0 to 15) and a column (0 to 31), so the next `printf` prints there: `PRINT@ n` is `locate(n / 32, n % 32)`.

4 Variables and numbers

In BASIC, a variable comes into being when you first use it, and holds a number with a fraction (or, with a `$`, a string). In C, you *declare* each variable first, at the start of a function, saying what *type* of value it holds:

```
int lives;      /* a whole number, -32768 to 32767 */
unsigned count;     /* a whole number, 0 to 65535 */
long distance;      /* a whole number, about -2 billion to 2 billion */
char letter;        /* a character (or a number, 0 to 255) */
float speed;        /* a number with a fraction, like BASIC's */
double precise;     /* the same, with more digits */
```

Everything between `/*` and `*/` is a *comment*, which the compiler ignores, like BASIC's `REM`.

Names can be long (BASIC looks only at the first two letters), and capitals matter: `score` and `Score` are different variables.

Why so many types? An `int` takes two bytes of memory and the 6809 adds two of them in a single instruction; a `float` takes four bytes and needs a whole routine to add. Most of a program's numbers are whole numbers, and `int` is the type for them. Use `long` when a number won't fit in an `int`, and `float` or `double` when you need fractions.

Watch the limits. An `int` can't hold more than 32767. Add 1 to 32767 and you get -32768, without any error. BASIC would have given you 32768.

4.1 Arithmetic

`+`, `-` and `*` are as in BASIC. `/` with two whole numbers divides and throws away the fraction: `7 / 2` is 3, and `%` gives the remainder, `7 % 2` is 1. With a `float` or `double`, `/` gives the fraction: `7.0 / 2` is 3.5.

`=` puts a value in a variable, as BASIC's `LET` does. C has shorthands for the common cases:

C	Means
`x = x + 1`	the same as in BASIC
`x += 5`	`x = x + 5` (and `-=`, `*=`, `/=`)
`x++`	`x = x + 1`
`x--`	`x = x - 1`

There is no operator for powers (C's `^` is something else); multiply (`x * x`), or use `pow()` from `math.h`. `math.h` also has `sqrt`, `sin`, `cos`, `atan`, `log`, `exp` and BASIC's other functions; `abs()` is in `stdlib.h`.

5 INPUT and INKEY$

BASIC's `INPUT` reads a line from the keyboard into a variable. In C you read the line as text, then turn it into a number:

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char *line = 0;
    size_t size = 0;
    int age;

    printf("HOW OLD ARE YOU? ");
    getdelim(&line, &size, '\r', stdin);
    age = atoi(line);
    printf("NEXT YEAR YOU WILL BE %d\r", age + 1);
    return 0;
}
```

`getdelim` reads what is typed, up to and including the `ENTER` key (`'\r'`), into a string that `line` points to; `atoi` turns the text at the start of the string into an `int`. For a number with a fraction, use `atof`, and a `float` or `double`. As in BASIC, the keys appear on the screen as they are typed, and the left arrow rubs out.

`line` and `size` keep the line and its size between calls, so call `getdelim` again with the same two to read the next line.

5.1 INKEY$

`inkey()`, from `coco.h`, is `INKEY$`: it gives the key being pressed, or 0 if none is, without waiting:

```
#include <stdio.h>
#include <coco.h>

int main()
{
    int k;

    printf("PRESS A KEY\r");
    while ((k = inkey()) == 0)
        ;
    printf("YOU PRESSED %c\r", k);
    return 0;
}
```

The `while` loop (next chapter) calls `inkey()` again and again until it gives something other than 0, like

```
10 K$=INKEY$:IF K$="" THEN 10
```

6 Deciding and repeating

6.1 IF

```
100 IF A > 10 THEN PRINT "BIG" ELSE PRINT "SMALL"
```

is

```
if (a > 10)
    printf("BIG\r");
else
    printf("SMALL\r");
```

The condition is always in parentheses. To do several things, enclose them in braces:

```
if (lives == 0) {
    printf("GAME OVER\r");
    return 0;
}
```

Comparing is ==, not =. `if (lives = 0)` *sets* `lives` to 0, and the compiler won't stop you. The comparisons are `==`, `!=` (BASIC's `<>`), `<`, `>`, `<=` and `>=`; `AND`, `OR` and `NOT` are `&&`, `||` and `!`:

```
if (x >= 0 && x <= 31)
    printf("ON THE SCREEN\r");
```

6.2 FOR

```
10 FOR I = 1 TO 10
20 PRINT I
30 NEXT I
```

is

```
for (i = 1; i <= 10; i++)
    printf("%d\r", i);
```

The three parts in the parentheses are what to do first (`i = 1`), the condition to go on (`i <= 10`), and what to do after each time round (`i++`). `STEP 2` is `i += 2`; counting down, `FOR I = 10 TO 1 STEP -1`, is `for (i = 10; i >= 1; i--)`.

6.3 Loops instead of GOTO

C has a `goto`, but you will rarely want it. A loop that goes on while something is true is a `while`:

```
while (lives > 0) {
    /* play */
}
```

and one that runs at least once is a `do`:

```
do {
    /* ask again */
} while (answer != 'Y' && answer != 'N');
```

Inside any loop, `break` leaves the loop at once, and `continue` goes on to the next time round. `for (;;)` loops for ever, until a `break` or a `return`.

6.4 ON...GOTO

```
100 ON C GOTO 200, 300, 400
```

becomes a `switch`, which chooses by a value:

```
switch (c) {
case 1:
    printf("ONE\r");
    break;
case 2:
    printf("TWO\r");
    break;
```

```
default:
    printf("SOMETHING ELSE\r");
    break;
}
```

Each `case` runs until a `break`; `default` is for any other value.

7 GOSUB: functions

In BASIC, a subroutine is lines you reach with `GOSUB` and leave with `RETURN`. In C, it is a *function*, with a name; `main` is one. A function can take values, and give one back:

```
#include <stdio.h>

int square(n)
int n;
{
    return n * n;
}

void stars(count)
int count;
{
    int i;

    for (i = 0; i < count; i++)
        printf("*");
    printf("\r");
}

int main()
{
    int i;

    for (i = 1; i <= 5; i++)
        printf("%d SQUARED IS %d\r", i, square(i));
    stars(10);
    return 0;
}
```

`square` takes an `int` called `n`, and gives back (`returns`) `n * n`. `stars` gives nothing back (`void`), and prints a row of stars. A function is written before the functions that call it.

The variables a function declares, such as each function's `i` here, are its own: changing one doesn't change another function's of the same name. That is unlike BASIC, where every variable is shared, and it is what lets you write a function without worrying about the rest of the program. A variable declared outside every function, before them, is shared by all of them, as BASIC's are.

(There is another way to write a function's heading, which most books use: `int square(int n)`. bcc09 takes both.)

8 Arrays

`DIM A(10)` makes an array of eleven numbers, `A(0)` to `A(10)`. In C:

```
int a[10];
```

makes ten, `a[0]` to `a[9]`: the number is how many, and they count from 0. C doesn't check that you stay inside an array; `a[10]` would be whatever is next in memory, so be careful at the ends.

Arrays can have more than one dimension, and can be given their values when they are declared:

```
int board[8][8];
int primes[5] = { 2, 3, 5, 7, 11 };
```

`board[2][3]` is row 2, column 3.

9 Strings

C has no string type like BASIC's `A$`. A string is an array of characters, ending with a 0, and the string functions in `string.h` do what BASIC's string operators do:

BASIC	C
`DIM A$` (no limit)	`char a[40];` (room for 39 characters and the 0)
`A$ = "HELLO"`	`strcpy(a, "HELLO");`
`A$ = A$ + B$`	`strcat(a, b);`
`LEN(A$)`	`strlen(a)`
`IF A$ = B$`	`if (strcmp(a, b) == 0)`
`LEFT$(A$, N)`	`strncpy(b, a, n); b[n] = 0;`
`MID$(A$, P, N)`	`strncpy(b, a + p - 1, n); b[n] = 0;`
`RIGHT$(A$, N)`	`strcpy(b, a + strlen(a) - n);`
`INSTR(A$, B$)`	`strstr(a, b)` (a pointer to it, or 0)
`VAL(A$)`	`atoi(a)` or `atof(a)`
`STR$(N)`	`sprintf(a, "%d", n);`
`ASC(A$)`	`a[0]`
`CHR$(N)`	`n`, as a character

The array must have room for what you put in it: unlike BASIC, C doesn't make it bigger, and writing past its end overwrites other variables.

A single character is written in single quotes, `'A'`, and is a number, the character's code (`'A'` is 65), so `if (k == 'Y')` compares a key with `Y`. `toupper()`, from `ctype.h`, turns a letter into a capital.

10 PEEK, POKE and the machine

`peek(a)` and `poke(a, v)`, from `coco.h`, are BASIC's `PEEK` and `POKE`:

```
poke(1024, 42);      /* a star (in reverse) at the screen's top left */
x = peek(65280);
```

C's own way to reach memory is a *pointer*, a variable holding an address. `peek` and `poke` are pointers in disguise:

```
unsigned char *screen;

screen = (unsigned char *)0x400;
screen[0] = 42;      /* the same as poke(0x400, 42) */
```

`0x400` is hexadecimal, as `&H400` is in BASIC: 1024.

10.1 The joysticks

`joystk(n)` is `JOYSTK(n)`: 0 to 63, `n` 0 and 1 for the right joystick's left-right and up-down, 2 and 3 for the left one's.

10.2 SOUND

`sound(tone, duration)`, from `coco.h`, is `SOUND`: the tone is 1 (low) to 255 (high), with BASIC's own pitches (89 is middle C), and the duration is in fifteenths of a second, as on the CoCo:

```
#include <coco.h>

int main()
{
    int t;

    for (t = 1; t <= 255; t += 20)
        sound(t, 1);
    sound(89, 15);          /* middle C, for a second */
    return 0;
}
```

10.3 Random numbers

`rand()`, in `stdlib.h`, gives a random whole number from 0 to 32767; `rand() % 6 + 1` is a die's throw, as `RND(6)` is. It gives the same numbers every time the program runs, unless you start it somewhere else with `srand()`: a good start is the time the player takes to press a key.

10.4 Faster variables

The 6809 has a *direct page*: 256 bytes of memory it can reach with a one-byte address, quicker than anywhere else. Color BASIC keeps its own busiest variables there, in page 0. A C program can put its own there by declaring them `__direct` (two underscores), outside any function:

```
#include <stdio.h>

__direct int i, total;

int main()
{
    for (i = 0; i < 100; i++)
        total += i;
    printf("TOTAL %d\r", total);
    return 0;
}
```

Each use of `i` or `total` is then a byte smaller and a little faster. Keep it for the variables a program uses most, such as a game's positions and counters: all of them together must fit in the 256 bytes. They get a page of their own, not BASIC's, and the library gives the ROM page 0 back whenever it calls it. If you call the ROM's routines from machine language yourself, see the bcc09 manual first.

10.5 What C doesn't have yet

bcc09 has no `PLAY`, `LINE`, `CIRCLE` or `PMODE` for you: C programs make their own music and graphics, by writing to the hardware and to the screen's memory, which is how the fastest games do it. The bcc09 manual says how to call machine language from C, and so the ROM's routines.

11 A complete program

A game of guessing a number, using most of what this guide has shown:

```
#include <stdio.h>
#include <stdlib.h>
#include <coco.h>

char *line = 0;
size_t size = 0;

/* ask for a number, until one is typed (-1 for BREAK) */
int ask(prompt)
char *prompt;
{
    for (;;) {
```

```
        printf("%s", prompt);
        if (getdelim(&line, &size, '\r', stdin) <= 0)
            return -1;
        if (line[0] >= '0' && line[0] <= '9')
            return atoi(line);
        printf("A NUMBER, PLEASE\r");
    }
}

int main()
{
    int secret, guess, tries;
    unsigned wait = 0;

    cls(-1);
    locate(0, 8);
    printf("GUESS MY NUMBER\r\r");
    printf("PRESS A KEY TO START");
    while (inkey() == 0)
        wait++;
    srand(wait);
    secret = rand() % 100 + 1;
    tries = 0;
    printf("\r\rI AM THINKING OF A NUMBER\r");
    printf("FROM 1 TO 100.\r");
    do {
        guess = ask("YOUR GUESS? ");
        if (guess < 0) {
            printf("\rBYE\r");
            return 0;
        }
        tries++;
        if (guess < secret)
            printf("TOO SMALL\r");
        else if (guess > secret)
            printf("TOO BIG\r");
    } while (guess != secret);
    printf("YES! IN %d TRIES\r", tries);
    return 0;
}
```

- `line` and `size` are declared outside the functions, so `ask` keeps them from one call to the next.
- `srand(wait)` starts the random numbers from how long the player took to press a key, so each game is different.
- `do ... while` asks until the guess is right.
- `getdelim` gives -1 when the player presses `BREAK` at the start of a line, and then the game ends.

Compile it, put it on a cassette, and load it:

```
bcc09 -Me -O -o guess -X-T0x3000 guess.c
objcopy09 -O cas -n GUESS guess guess.cas
```

12 A game

`worm.c`, which comes with bcc09's documentation (in `/usr/share/doc/bcc09-doc` from the RPM, or `doc/worm.c` in its source), is a game of the kind people wrote in BASIC and wished ran faster: a worm crawls across the screen, the arrow keys turn it, and each piece of food makes it longer and faster, until it runs into the wall or itself. It is about 200 lines, and uses what this guide has shown:

- **The screen is the board.** The worm, the wall and the food are blocks of colour in the text screen's memory, `CHR$(128 + 16 * colour + 15)` as BASIC would print them, put there with `poke`. To find out what is ahead of the worm, the program `peek`s at it:

```
#define AT(p)   peek(SCREEN + (p))
#define PUT(p, c)   poke(SCREEN + (p), (c))
...
        p = worm[head] + step;
        if (AT(p) == FOOD) {
            ...
        } else if (AT(p) != EMPTY) {
            sound(20, 6);   /* the wall, or itself */
            return;
        }
```

- **One key a move,** with `inkey()`, as a BASIC game reads `INKEY$` once each time round its loop. The arrow keys give 94 (up), 10 (down), 8 (left) and 9 (right), as in BASIC.
- **The worm is an array** of the screen positions of its pieces, used as a ring: the head is added at one end and the tail taken off the other, and the indexes are `unsigned char`s, which count from 255 back to 0 by themselves.
- **The speed** is an empty `for` loop between moves, which gets shorter as the worm grows; in C it has to be a long one (3500 times round, at the start) to slow the worm down to about 8 moves a second.
- **The score** is printed on the top line with `locate()` and `printf()`; `fflush(stdout)` makes it appear without a `\r`.

Build it as any other program, and put it on a cassette:

```
bcc09 -Me -O -o worm -X-T0x3000 worm.c
objcopy09 -O cas -n WORM worm worm.cas
```

It is under the ISC license, not the GPL: change it, or start a game of your own from it.

13 Cartridges and disks

A ROM cartridge. `bcc09 -Mr` makes a cartridge's image, which the CoCo 1 and 2 and the Dragon start as soon as they are switched on, for an EPROM or for XRoar:

```
bcc09 -Mr -O -o guess.rom guess.c
xroar -cart-rom guess.rom -cart-autorun
```

The same program works, `coco.h` and all. When `main` returns, the program stops, since a cartridge has no BASIC to go back to.

Disk files. With Disk BASIC on a CoCo, a program can read and write files, through `fopen`, `fprintf`, `getdelim` and `fclose`:

```
#include <stdio.h>

int main()
{
    FILE *f;

    f = fopen("SCORES.DAT", "w");
    if (f == 0) {
        printf("CAN'T OPEN SCORES.DAT\r");
        return 1;
    }
    fprintf(f, "1200\r");
    fclose(f);
    return 0;
}
```

The files are Disk BASIC's, so BASIC can read what the program wrote, with `OPEN "I",#1,"SCORES.DAT"` and `INPUT #1`.

14 Where next

- **The bcc09 user's manual** (`manual.pdf`) describes the compiler, the C it takes, each kind of program it makes (Color BASIC, cartridges, OS-9, FLEX, the Vectrex), the C library, and how C and machine language call each other.
- **The manual pages**, `man bcc09` and the others, have every option.
- **A book on C** teaches the rest of the language: pointers, structures, and the standard library. Its programs end lines with `\n`; on the CoCo, end them with `\r`.

Appendix: BASIC to C

BASIC	C
`REM`	`/* ... */`
`LET A = 5`	`a = 5;`
`PRINT "HI"`	`printf("HI\r");`
`PRINT A`	`printf("%d\r", a);`
`PRINT A;`	`printf("%d", a);`
`PRINT@ N, "HI"`	`locate(n / 32, n % 32); printf("HI");`
`CLS`, `CLS C`	`cls(-1);`, `cls(c);`
`INPUT A`	`getdelim(&line, &size, '\r', stdin); a = atoi(line);`
`A$ = INKEY$`	`a = inkey();`
`IF A = B THEN ...`	`if (a == b) ...`
`IF A <> B`	`if (a != b)`
`AND`, `OR`, `NOT`	`&&`, `||`, `!`
`FOR I = 1 TO N ... NEXT`	`for (i = 1; i <= n; i++) { ... }`
`GOSUB 1000 ... RETURN`	a function: `name(); ... return;`
`ON N GOTO ...`	`switch (n) { case 1: ... }`
`DIM A(10)`	`int a[11];`
`A$ + B$`	`strcat(a, b)`
`LEN(A$)`	`strlen(a)`
`VAL(A$)`, `STR$(N)`	`atoi(a)`, `sprintf(a, "%d", n)`
`PEEK(A)`, `POKE A, V`	`peek(a)`, `poke(a, v)`
`JOYSTK(N)`	`joystk(n)`
`SOUND T, D`	`sound(t, d);`
`RND(N)`	`rand() % n + 1`
`INT(X)`	`floor(x)` (`math.h`)
`ABS(X)`, `SQR(X)`	`abs(x)`, `sqrt(x)`
`END`	`return 0;` (in `main`)